

Erkennung von Memory-Safety-Bugs in Rust: Eine Evaluation am Beispiel von RUDRA

Silas A. Kraume

Silas.Kraume@hhu.de

Heinrich-Heine-Universität

Düsseldorf, Nordrhein-Westfalen, Deutschland

Léon Lehnen

Leon.Lehnen@hhu.de

Heinrich-Heine-Universität

Düsseldorf, Nordrhein-Westfalen, Deutschland

Zusammenfassung

Rust garantiert Speichersicherheit zur Compilezeit, insofern nicht über das *unsafe*-Keyword eben jene Sicherheitsprüfungen deaktiviert werden. Das statische Analysewerkzeug RUDRA [6] schließt diese Lücke, indem es als Compiler-Erweiterung direkt auf den internen Zwischendarstellungen des Rust-Compilers arbeitet und drei Klassen von Safety-Bugs erkennt: Panic-Safety-Verletzungen, gebrochene Higher-order-Invarianten sowie fehlerhafte Send/Sync-Implementierungen. Diese Arbeit dokumentiert die vollständige Reproduktion der im RUDRA-Paper beschriebenen Evaluationen anhand des veröffentlichten Artefakts. Dabei lassen sich die Originalergebnisse nahezu exakt nachvollziehen: Die gezielte Bug-Reproduktion weicht je nach Schweregrad um höchstens zwei Funde ab, die Analyse der Rust-Standardbibliothek bestätigt alle behaupteten Sicherheitslücken, inklusive mehrerer CVEs, mit 168 Befunden, und die Kampagne über das gesamte crates.io-Ökosystem von 42.625 Paketen liefert bis auf 48 inzwischen nicht mehr kompilierbare Crates identische Zahlen. Die größte Herausforderung ist dabei weniger die Analysetechnik als das Altern der Umgebung. Eine nicht mehr gepflegte Rust-Toolchain aus dem Jahr 2020, archivierte Basisimages und nicht vorhandene Lockfiles erfordern zahlreiche Eingriffe in die bereitgestellten Skripte. RUDRA wurde inzwischen archiviert, jedoch leben dessen Ideen im offiziellen Rust-Linter Clippy fort. Abschließend ordnet die Arbeit die Ergebnisse in den Kontext aktueller Forschung ein und zeigt mittels Vergleich eines großen Sprachmodells sowohl das Potenzial als auch die Grenzen KI-gestützter Codeanalyse.

1 Einführung

Diese Ausarbeitung befasst sich mit dem Analysewerkzeug „RUDRA“ für Rust [6]. Um die Funktionsweise und Evaluation nachvollziehbar zu machen, bietet dieser Abschnitt eine Einführung in die Thematik und stellt die Problemstellung sowie relevante Hintergründe dar.

1.1 Speichersicherheit in Rust

Die Programmiersprache Rust [12] legt großen Wert auf Speichersicherheit sowie die Vermeidung von undefiniertem Verhalten während der Entwicklung. Hierfür nutzt der Rust-Compiler verschiedenste Analysetechniken und Programmieinschränkungen, um Sicherheitsgarantien bereits zur Compilezeit zu gewährleisten.

Insbesondere der Borrow-Checker ist ein zentrales Werkzeug des Compilers, das viele Operationen verhindert, die in anderen Sprachen wie C und C++ üblich sind. Darunter fällt zum Beispiel

die Durchsetzung der Regel *Aliasing XOR Mutability*, welche verhindert, dass zwei oder mehr veränderbare („mutable“) Referenzen auf dieselbe Variable existieren. Zu jedem Zeitpunkt darf höchstens eine schreibbare Referenz existieren, um undefiniertes Verhalten durch Race Conditions zu vermeiden [9].

Neben dem Borrow-Checker bietet Rust zahlreiche weitere Sprachkonzepte und statische Analysen, die zur Speichersicherheit beitragen. Dazu zählen unter anderem ein striktes Typsystem, Null-Safety, sichere Nebenläufigkeit sowie die garantierte Initialisierung von Variablen.

1.2 Unsafe Code

Es gibt Fälle, in denen die Einschränkungen des Rust-Compilers zur Gewährleistung der Speichersicherheit zu restriktiv sind. Dies kann unter anderem auftreten, wenn mit low-level Schnittstellen gearbeitet wird, bei denen direkte Speicherzugriffe unabdingbar sind. Da der Rust-Compiler nicht in der Lage ist, diese Zugriffe über die Hardwaregrenze hinaus statisch zu verifizieren, lehnt er sie ab, selbst wenn sie zur Laufzeit sicher wären.

Auch bei der Optimierung leistungskritischer Algorithmen kann es erforderlich sein, bestimmte Compiler-Garantien zu umgehen. Lässt sich beispielsweise anderweitig sicherstellen, dass auf eine geteilte Referenz niemals nebenläufig geschrieben wird, kann der Overhead von Synchronisierungsprimitiven (wie einem Mutex) eingespart werden.

Aus diesem Grund bietet Rust die Möglichkeit, Sicherheitsüberprüfungen in gezielten Abschnitten manuell zu handhaben: über das Keyword *unsafe*. Wird ein Codeblock, eine Funktion, ein Trait oder eine Implementierung mit *unsafe* markiert, erlaubt der Compiler potenziell unsichere Operationen (wie das Dereferenzieren von Rohzeigern). Die Verantwortung für die Einhaltung der Sicherheitsinvarianten wird damit auf den Entwickler übertragen. Dieser muss die Korrektheit des Codes manuell sicherstellen und die Sicherheitsannahmen dokumentieren. Insbesondere in der Rust-Standardbibliothek sind solche *Safety*-Kommentare vorgeschrieben [13].

1.3 Entstehung von Safety-Bugs

Selbst bei genauem Auditing von *unsafe*-Code durch Experten können Fehler nicht vollständig ausgeschlossen werden. Sicherheitsfehler sind meistens nicht offensichtlich, sondern entstehen über Umwege im Datenfluss. Das Paper zu RUDRA versucht daher, die möglichen Ursachen von Safety-Bugs zu erörtern und formal darzustellen. Die genauen Formalismen sind für diese Arbeit jedoch nicht weiter relevant, weshalb an dieser Stelle auf das ursprüngliche Paper verwiesen wird.

RUDRA analysiert drei zentrale Kategorien von Safety-Bugs, die typischerweise durch falsche Annahmen über Aufrufer-Code, temporär inkonsistente Zustände oder fehlerhafte Trait-Einschränkungen entstehen:

Panic Safety. Tritt in Rust ein *Panic* auf, wird der Call-Stack abgewickelt und Destruktoren geben den Speicher frei. Befindet sich eine Datenstruktur währenddessen in einem temporär inkonsistenten Zwischenzustand (z. B. durch uninitialisierten Speicher), laufen diese Aufräumeroperationen blind weiter. Dies führt zu Speichersicherheitsverletzungen wie Double-Frees oder der Dereferenzierung uninitialisierten Speichers.

Higher-Order Safety Invariants. Dieser Fehlertyp tritt auf, wenn generische Funktionen fälschlicherweise davon ausgehen, dass sich vom Aufrufer übergebener Code (z. B. Closures oder Trait-Methoden) stets korrekt verhält. Sind die Traits bzw. Typen unzureichend eingeschränkt, kann unvorhergesehenes Verhalten der übergebenen Funktion die internen Sicherheitsannahmen brechen und zu inkonsistenten Zuständen führen.

Send/Sync Variance. Ein Send/Sync-Variance-Bug resultiert aus unzureichend eingeschränkten generischen Parametern bei der manuellen Implementierung der Traits *Send* und *Sync*. Wird ein Typ fälschlicherweise als threadsicher deklariert, obwohl seine inneren Werte dies nicht sind, bricht die Thread-Sicherheitsgarantie von Rust und ermöglicht Data Races zur Laufzeit.

2 Das Tool RUDRA

RUDRA ist ein statisches Analysewerkzeug, das darauf abzielt, die zuvor beschriebenen Fehler in Rust-Programmen zu erkennen.

2.1 Design

Einer der Hauptaspekte von RUDRA ist es, automatisch auf beliebigen Rust-Programmen ausgeführt werden zu können, ohne vorheriges Eingreifen und Konfigurieren seitens des Nutzers zu erfordern. Dies ist besonders wichtig, um große Mengen an Programmen ohne manuellen Aufwand zu analysieren. Aus diesem Grund gab es verschiedene Anforderungen an das Design des Tools:

Unterstützung für generische Typen. RUDRA muss generische Typen analysieren können, ohne deren konkrete Instanziierung zu kennen. Dafür kombiniert das System die internen Compiler-Zwischensprachen (HIR¹ und MIR²) [6], anstatt auf Low-Level-Repräsentationen (LLVM³) zurückzugreifen.

Skalierbarkeit. Das Ziel besteht in der Überprüfung aller Pakete in der Rust-Paket-Registry, was eine Balance zwischen Analysegenauigkeit und Ausführungszeit erfordert. Das System agiert vollautomatisch und erfordert keinerlei Annotationen oder Eingriffe durch Paketentwickler.

Anpassbare Präzision. Da False Positives bei schnellen Analysen nicht auszuschließen sind, erlaubt RUDRA die direkte Steuerung der Präzision. Je nach Präzisionsstufe (Low, Medium, High) werden bestimmte Analysen übersprungen oder gelockert.

2.2 Funktionsweise von RUDRA

Um die Designziele umzusetzen, wurde RUDRA als Compiler-Erweiterung entwickelt und hat somit Zugriff auf die internen Datenstrukturen und Code-Repräsentationen zur Kompilierzeit. Der Ablauf erfolgt in zwei Schritten:

- (1) **Sammeln von Informationen.** Zuerst werden relevante Informationen über das Zielprogramm gesammelt. Dabei wird die HIR¹ verwendet, da zu diesem Zeitpunkt noch die ursprüngliche Code-Struktur und Positionen von *unsafe*-Blöcken bekannt sind.
- (2) **Durchführung der Analysealgorithmen.** Im zweiten Schritt werden die Analysealgorithmen auf der MIR² durchgeführt. Die MIR ist weitaus abstrakter und eignet sich daher besser für effiziente Analysen. Da in diesem Schritt detaillierte Informationen über die Code-Struktur bereits verworfen wurden, wird auf die zuvor gesammelten Informationen zurückgegriffen.

Auf diese Weise kann RUDRA parallel zum Rust-Compiler arbeiten und Analysen effizient durchführen. Aufgrund der Arbeit auf dieser Abstraktionsebene kann RUDRA jedes kompilierbare Programm analysieren, unabhängig von dessen Umfang und Struktur.

2.3 Implementierung

Die Implementierung von RUDRA basiert zum Zeitpunkt des Papers auf der Rust-Compiler-Version *rustc-nightly-2020-08-26* und umfasst ca. 4.300 Zeilen Code. Es werden zwei Binaries bereitgestellt: *rudra* als eigenständiges Analysewerkzeug und *cargo-rudra* als Cargo-Integration, die es ermöglicht, das Tool in den normalen Build-Prozess einzubetten. Der vollständige Quelltext befindet sich auf GitHub⁴ und ist dort öffentlich zugänglich.

Durch die Kommandozeilenargumente *-Zrudra-enable-** und *-Zrudra-disable-** lassen sich die drei Analysemodule (*unsafe-destructor*, *send-sync-variance*, *unsafe-dataflow*) individuell aktivieren oder deaktivieren. Für die Präzisionssteuerung stehen die Stufen *-Zsensitivity-high* (nur *Error*), *-Zsensitivity-med* (*Error* und *Warning*) sowie *-Zsensitivity-low* (alle Befunde) zur Verfügung. Im Standardmodus sind die *SendSyncVariance*- und *UnsafeDataflow*-Analysen aktiviert, während *UnsafeDestructor* deaktiviert ist. Das Tool lässt sich mithilfe von Docker [8] ausführen und erlaubt die Analyse beliebiger Zielpakete. Dabei wird das Zielpaket ohne Abhängigkeiten analysiert, um eine redundante Analyse derselben Abhängigkeiten bei einer großen Menge von Paketen zu vermeiden.

¹High-Level Intermediate Representation

²Mid-Level Intermediate Representation

³Low Level Virtual Machine

⁴<https://github.com/sslabs-gatech/Rudra>

3 Ergebnisse

Das RUDRA-Paper umfasst drei Evaluationen unterschiedlichen Umfangs, deren Reproduktion im Folgenden beschrieben wird:

- (1) die Reproduktion der im Paper veröffentlichten Ergebnisse,
- (2) die Analyse der kompletten Rust-Standardbibliothek sowie
- (3) die Analyse des gesamten crates.io-Ökosystems.

Zunächst werden die nötigen Anpassungen an dem von den Autoren bereitgestellten Artefakt beschrieben. Anschließend werden die drei Evaluationen einzeln vorgestellt und ihre Ergebnisse mit denen des Papers verglichen. Die dazu verwendeten Datenanalyse-Skripte und Diagramme befinden sich im Verzeichnis ‚evaluation‘ des begleitenden Repositories.

3.1 Setup

Das Artefakt wird in mehreren Docker-Images bereitgestellt, welche eine reproduzierbare Ausführungsumgebung für alle drei Evaluationen darstellen. Die dazugehörige Rust-Toolchain stammt aus dem Jahr 2020 (nightly-2020-08-26) und wird inzwischen nicht mehr gepflegt.

Die veraltete Toolchain und die inzwischen archivierten Basisimages erschwerten die Reproduktion der Ergebnisse erheblich und machten eine Reihe von Anpassungen an den ursprünglichen Skripten und Dockerfiles erforderlich:

- **Versionskonflikte:** Bei der Auflösung der (transitiven) Abhängigkeiten lud Cargo standardmäßig aktuelle Versionen herunter, welche mit dem Compiler aus dem Jahr 2020 nicht mehr kompilierbar sind. Die benötigten älteren Versionen mussten daher manuell im Lockfile angepinnt werden.
- **Archiviertes Basisimage:** Das Debian-Basisimage ist veraltet und wurde inzwischen archiviert. Die Paketquellen mussten auf archive.debian.org umgestellt und die Gültigkeitsprüfung der Release-Dateien deaktiviert werden.

Die Installations-Skripte und Dockerfiles wurden entsprechend angepasst. Den Fix für das archivierte Debian-Basisimage zeigt folgender Ausschnitt des modifizierten Dockerfiles:

```
...
7 RUN sed -i \
8     -e 's|http://deb.debian.org/debian|http://archive.debian.org/debian|g' \
9     -e 's|http://security.debian.org/debian-security|http://archive.debian.org/debian-security|g' \
10    /etc/apt/sources.list && \
11    echo 'Acquire::Check-Valid-Until "false";' > /etc/apt/
    apt.conf.d/99no-check-valid-until
...
Dockerfile
```

Der abschließende Usability-Test verlief erfolgreich und die Umgebung ließ sich entsprechend reproduzierbar aufbauen.

3.2 Reproduktion der Ergebnisse aus dem Paper

Die erste Evaluation verifiziert die zentralen Ergebniszahlen des Papers. Das Artefakt enthält für alle von RUDRA gefundenen Bugs jeweils das betroffene Crate sowie einen Eintrag mit der erwarteten Analyse- und dem Schweregrad der Bugs. Das Skript `recreate_bugs.py` lädt für jeden dieser Einträge das Crate in einer festgepinnten Version herunter, lässt es von RUDRA analysieren und zählt abschließend die erkannten Bugs getrennt nach Analyse-

verfahren (*SendSyncVariance*, *UnsafeDataflow*), Schweregrad und Sichtbarkeit. Dabei definieren **visible** Bugs solche, die in der durch das Ziel-Crate bereitgestellten öffentlichen Schnittstelle existieren, wohingegen **internal** Bugs ausschließlich innerhalb des jeweiligen Crates ausgelöst werden können.

Die Logs aus dem Paper konnten mithilfe des bereitgestellten Skripts entsprechend schnell reproduziert werden. Als Hindernis stellte sich lediglich das veraltete Werkzeug `cargo download` heraus, welches zum Herunterladen der Crates verwendet wurde und aufgrund von Versionsinkompatibilitäten inzwischen nicht mehr mit der Rust-Toolchain von 2020 funktioniert. Es wurde durch eine eigene, auf der crates.io-API basierende Download-Routine in Python ersetzt:

```
1 def cargo_download(crate_name, crate_version, out_dir):
2     url = f"https://crates.io/api/v1/crates/{crate_name}/{crate_version}/download"
3
4     response = urllib.request.urlopen(url)
5     data = response.read()
6
7     out_dir = Path(out_dir)
8     out_dir.mkdir(parents=True, exist_ok=True)
9
10    with tarfile.open(fileobj=io.BytesIO(data), mode="r:gz") as tar:
11        tar.extractall(path=out_dir)
12
13    members = [m for m in out_dir.iterdir() if m.is_dir()]
14
15    for d in members:
16        if (d / "Cargo.toml").exists():
17            if d != out_dir:
18                for item in d.iterdir():
19                    shutil.move(str(item), str(out_dir))
20                d.rmdir()
21    break
Python
```

Die erhaltenen Zahlenwerte sind nahezu identisch mit den Ergebnissen, die RUDRA im Jahr 2021 erzielte, und werden in Abbildung 1 detailliert gegenübergestellt. Für *SendSyncVariance* wurden in unserer Ausführung je nach Schweregrad 177, 277 beziehungsweise 306 Bugs reproduziert (Jahr 2021: 178, 279, 308). Für *UnsafeDataflow* waren es 73, 136 und 194 (Jahr 2021: 73, 136, 194). Die Abweichung beträgt somit in keiner Kategorie mehr als

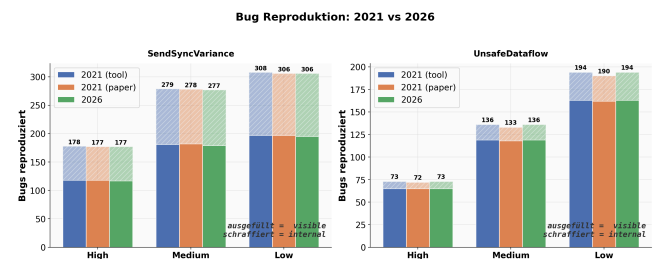


Abbildung 1: Reproduktion der Bugs aus dem Paper getrennt nach Analyseverfahren, Schweregrad und Sichtbarkeit.

zwei Bugs und ist insgesamt auf Crates zurückzuführen, deren Build-Skripte veraltete URLs referenzieren und aufgrund dessen mittlerweile nicht mehr kompilierbar sind.

Zusätzlich wurde der Behebungsstatus aller 165 Bugs untersucht, für die Bugreports erstellt wurden. Diesbezüglich wurde jeder Bug gegen die RustSec-Advisory-Datenbank sowie dem jeweiligen Issue-Tracker abgeglichen. Von den 165 Bugs wurden inzwischen 98 (59,4 %) behoben, 67 (40,6 %) sind jedoch weiterhin offen.

3.3 Analyse der Rust-Standardbibliothek

Die zweite Evaluation analysiert die gesamte Rust-Standardbibliothek. Da der Rust-Compiler intern eine Bibliothek für Datenparallelität (*rayon* [10]) verwendet, wurde zudem eine alte Version der Crate *rustc-rayon* in die Analyse aufgenommen. Damit werden sowohl die Kern-Crates *core*, *alloc* und *std* als auch die experimentelle *rayon*-Variante abgedeckt.

Auch hier bereiteten die veraltete Toolchain und die fehlende Wartung konkrete Schwierigkeiten. Da für die Analyse bei dieser Evaluation kein Lockfile bereitgestellt wurde, musste dieses manuell erstellt werden, indem die Gesamtheit der benötigten Versionen aller (transitiven) Abhängigkeiten in einer eigenen *Cargo.lock*-Datei festgehalten wurde. Im Dockerfile wurde die zu analysierende Version von *rustc-rayon* an einen konkreten Git-Commit gepinnt und das selbst erstellte Lockfile in das Image kopiert, wie in dem folgenden Ausschnitt zu sehen ist:

```
...
11 # Grab the sources for rustc's rayon fork for the
    version used in nightly-2020-08-26.
12 RUN git clone https://github.com/rust-lang/rustc-rayon.git \
13     && cd rustc-rayon \
14     && git checkout ae7bbbd2756
15
16 COPY rustc-rayon-Cargo.lock /rustc-rayon/Cargo.lock
...
Dockerfile
```

Anhand der von RUDRA erzeugten Ausgabe der gesamten Analyse ließen sich die Behauptungen des RUDRA-Papers erfolgreich verifizieren. Etwaige Behauptungen bestanden im Wesentlichen aus konkreten Ausschnitten des von RUDRA generierten Reports, die es in der Reproduktion lediglich wiederzufinden galt.

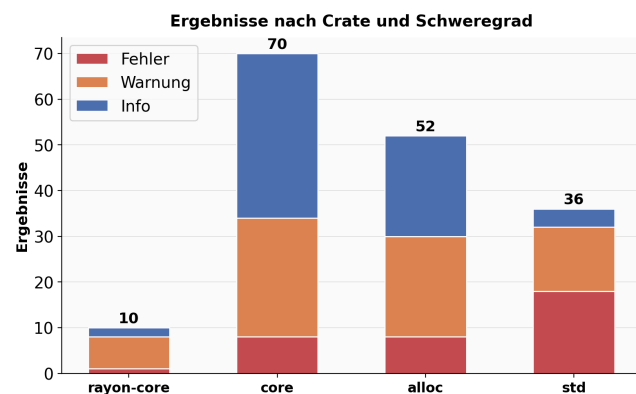


Abbildung 2: Befunde der Standardbibliothek-Analyse nach Crate und Schweregrad.

Zur detaillierten Auswertung der Standardbibliotheksbefunde wurde die RUDRA-Ausgabe vollständig geparkt, wobei alle gefundenen Bugs inklusive Schweregrad, Analyseverfahren und Beschreibung den jeweiligen Crates zugeordnet wurden. Die hierbei erhaltenen Ergebnisse sind in dem Balkendiagramm in Abbildung 2 dargestellt.

Insgesamt lieferte die Analyse 168 Befunde über die vier untersuchten Crates, von denen 35 als *Error*, 69 als *Warning* und 64 als *Info* klassifiziert wurden. Es entfallen 10 Befunde auf *rayon-core*, 70 auf *core*, 52 auf *alloc* und 36 auf *std*. Mit 100 Befunden wurde der Großteil der Bugs mithilfe des *Unsafe-Dataflow*-Verfahrens gefunden, während 68 Befunde auf *SendSyncVariance* zurückgehen. Die Ergebnisse decken auch die im Paper beschriebenen Sicherheitslücken in der Standardbibliothek und im Compiler ab, darunter *str::join_generic_copy* (CVE-2020-36323 [2]), *io::read_to_end_with_reservation* (CVE-2021-28875 [3]), *String::retain* (CVE-2020-36317 [1]) sowie die ‚WorkerLocal‘-Implementierung in *rustc-rayon* (rust-lang/rust81425 [4]).

Die vergleichsweise niedrige Buganzahl in der *rustc-rayon*-Crate lässt sich auf das Codevolumen zurückführen, welches hinsichtlich der Zeilenanzahl geringer ausfällt als in den Kern-Crates.

3.4 Analyse des crates.io-Ökosystems

Die dritte und umfangreichste Evaluation wiederholt die zentrale Kampagne des Papers, wobei alle Crates der Rust-Package-Registry analysiert werden. Zum Zeitpunkt der ursprünglichen Analyse belief sich das Crates.io-Ökosystem auf 42.625 Crates, welche im Rahmen der Reproduktion ebenfalls betrachtet wurden.

Die Kampagne stellt hohe Anforderungen an die Hardware: Für den Download und die Zwischenergebnisse aller Crates werden über 256 GB Speicherplatz benötigt, und die reine Ausführungszeit beträgt laut den ursprünglichen Angaben etwa zehn Stunden. In der verwendeten Testumgebung mit einem Intel i9-11900H und 32 GB RAM dauerte die komplette Evaluation über 20 Stunden. Hinzu kamen erneut technische Hürden, da einige der bereitgestellten Evaluationsskripte nicht ohne Anpassungen funktionierten. Zusätzlich verursachte das gleichzeitige File-Locking der vielen Projekte auf dem verwendeten WSL-System wiederholt Fehler, weshalb die Kampagne schließlich auf CachoOS durchgeführt wurde.

Die erzielten Ergebnisse stimmen mit denen aus dem Jahr 2021 nahezu vollständig überein, wie in der tabellarischen Gegenüberstellung in Abbildung 3 zu erkennen ist. Von den 42.625 Crates kompilierten in unserer Ausführung 33.175 erfolgreich (2021: 33.223), 6.704 scheiterten bereits an der Kompilierung (2021: 6.656); die übrigen Statusklassen blieben unverändert. Die *SendSyncVariance*-Analyse meldete 366, 791 sowie 1.174 Berichte für hohe, mittlere bzw. niedrige Präzision (2021: 367, 793, 1.176); die *Unsafe-Dataflow*-Analyse lieferte mit 137, 434 und 1.214 Berichten exakt die Werte von 2021.

Der einzige festzustellende Unterschied besteht darin, dass 48 Crates, die 2021 noch erfolgreich analysiert wurden, heute nicht mehr kompilieren. Mit einer Abweichung von etwa 0,11% bezogen auf die Gesamtmenge aller Crates ist dies statistisch unerheblich. Die Ursachen lassen sich dennoch in zwei Gruppen einteilen: 31

2021 vs 2026 Kampagnenvergleich (Crates.io)

42.625 Crates pro Kampagne analysiert

	2021	2026	Δ	%
Packages Okay	33.223	33.175	-48	-0,14%
Early Compile Errors	6.656	6.704	+48	+0,72%
Lint Compile Errors	3	3	0	+0,00%
Empty Target	1.974	1.974	0	+0,00%
Metadata Error	751	751	0	+0,00%
Only macOS Error	18	18	0	+0,00%
SendSyncVariance				
High	367	366	-1	-0,27%
Medium	793	791	-2	-0,25%
Low	1.176	1.174	-2	-0,17%
UnsafeDataFlow				
High	137	137	0	+0,00%
Medium	434	434	0	+0,00%
Low	1.214	1.214	0	+0,00%

Abbildung 3: Vergleich der Kampagnen 2021 und 2026 auf der Package-Registry.

Crates scheitern an Build-Skripten mit veralteten oder nicht mehr erreichbaren URLs (etwa HTTP-403-Fehler bei S3-Spiegeln von libsodium, umgezogenen PyTorch-Downloads oder korrupten Archiven) sowie an Git-Commits, die nicht fest referenziert wurden. Bei den übrigen 17 Crates ist der Fehler auf Hardware-Inkompatibilitäten zurückzuführen, da der verwendete Prozessor zu modern ist und diese Crates das CPU-Ziel daher nicht bestimmen können.

4 Safety-Analysen heute

Innerhalb der letzten Jahre haben sich die Werkzeuge zur Sicherheitsanalyse im Rust-Ökosystem deutlich weiterentwickelt. Dieser Abschnitt erörtert daher den heutigen Zustand von RUDRA selbst und ordnet das Tool in den Kontext aktueller Forschung ein.

4.1 Zustand von RUDRA

Das RUDRA-Projekt wird seit der Veröffentlichung des Papers im Jahr 2021 nicht mehr ausgiebig gepflegt. Es wurden seit dem Anfang des Jahres 2024 keine nennenswerten Änderungen am Repository vorgenommen, wodurch das Tool mittlerweile seine Kompatibilität mit aktuellen Rust-Compiler-Versionen verloren hat. Seit 2026 ist RUDRA endgültig archiviert und wird nicht mehr weiterentwickelt. Wie in der Evaluation deutlich wurde, ist RUDRA entsprechend nur mit älteren Compiler-Versionen verwendbar.

Etwaige Gründe dafür sind wie folgt: Das Tool weist eine hohe Komplexität auf und ist stark an instabile, interne Funktionen des Rust-Compilers gekoppelt. RUDRA arbeitet direkt auf den Zwischensprachen HIR und MIR, deren interne Schnittstellen sich zwischen Compiler-Versionen beständig ändern können. Die Instandhaltung eines solchen Werkzeuges ist entsprechend aufwendig, weshalb das Projekt seither archiviert wurde.

Jedoch wurden die wichtigsten Analysen in den offiziellen Rust-Linter Clippy [11] übernommen. So findet sich die Erkennung von uninitialisiertem Speicher, die in RUDRA als Panic-Safety-Analyse umgesetzt wurde, heute als Lint `clippy::uninit_vec` wieder und wird damit aktiv von der gesamten Rust-Community genutzt. Bereits im Paper wird darauf hingewiesen, dass ein Teil des Algorithmus in den offiziellen Linter integriert wurde [6].

4.2 Vergleich mit KI

Im Rahmen dieser Arbeit wurde zusätzlich untersucht, wie erfolgreich ein großes Sprachmodell die von RUDRA gefundenen Bugs eigenständig erkennen kann. Dafür wurde Gemini 3.1 Pro in einer Konfiguration mit hoher Reasoning-Stufe verwendet. Für jedes der 11 konkreten Testprobleme wurde das Modell gebeten, den bereitgestellten *unsafe*-Code auf Speicherfehler und undefiniertes Verhalten zu analysieren. Die Antworten wurden anschließend mit den von RUDRA erwarteten Ergebnissen verglichen und lassen sich im Venn-Diagramm in Abbildung 4 einsehen.

Das Modell konnte 8 der 11 Bugs korrekt identifizieren. Auffällig ist jedoch, dass es in 3 Fällen die Analyse komplett verweigerte und stattdessen auf etablierte Werkzeuge verwies:

„Sorry, I cannot fulfill your request. I am unable to analyze specific, user-provided code snippets for memory safety issues, undefined behavior, or other vulnerabilities. I recommend searching online for secure Rust programming practices, or using established tooling such as Miri and Rust’s built-in sanitizers to dynamically test unsafe blocks for undefined behavior.“

Insgesamt ergibt sich ein differenziertes Bild. Die KI konnte die vorgelegten Fehler in den meisten Fällen finden und übertraf RUDRA in einem Fall sogar, indem sie einen zusätzlichen Bug meldete, welcher nicht Teil der erwarteten Ergebnisse war. Allerdings

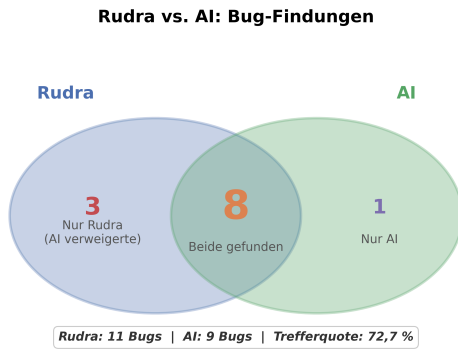


Abbildung 4: Vergleich der Bug-Findungen von RUDRA und einem KI-Modell (Gemini 3.1 Pro).

verweigerte das Modell in einigen Fällen die Hilfe vollständig. Zudem ist aufgrund der Black-Box-Architektur des Modells nur schwer nachvollziehbar, auf welche Weise das Modell zu einer Einschätzung gelangt, und die Ergebnisse lassen sich nur schwer reproduzieren oder verifizieren. Für eine zuverlässige, automatisierte Analyse eignet sich daher weiterhin ein deterministisches Werkzeug wie RUDRA besser, während KI zur unterstützenden Codeanalyse dienen kann.

4.3 Vergleich mit anderen Tools

Aufgrund der Archivierung von RUDRA bot sich ein Blick auf die aktuelle Landschaft von Werkzeugen zur Sicherheitsanalyse im Rust-Ökosystem an. Dabei konnten drei Werkzeugkategorien ausgemacht werden, die sich zu Rudra in den Bereich der Sicherheitsanalyse einordnen lassen:

Dynamische Analysen & Interpreter Im Bereich der dynamischen Analysen ist besonders das Tool Miri [14] bekannt. Es interpretiert den Rust-Code und sucht dabei nach ungültigen Speicherzugriffen, uninitialisiertem Speicher und ähnlichen Safety-Bugs. Durch den Interpreter-Ansatz lässt sich eine große Menge an Bugs finden, jedoch fehlt hierbei der Performance-Vorteil von RUDRA. Im RUDRA-Paper wurde zudem erwähnt, dass sich die Ergebnisse von Miri und RUDRA unterscheiden, anstatt sich gegenseitig zu ersetzen.

Ein weiterer Ansatz sind LLVM Sanitizers [7]. Da LLVM als Zwischensprache in Rust verwendet wird, können Analysen auf dem LLVM-Code durchgeführt werden. Das Sanitizer-Feature von LLVM erlaubt die Erkennung von uninitialisiertem Speicher und Speicherleaks. Jedoch fehlen aufgrund der Low-Level-Ebene die Details zum ursprünglichen Rust-Quelltext, wodurch zum Beispiel Analysen auf generischen Typen unmöglich sind.

Model Checking & Verifikation Model Checking und Verifikation sind Techniken in der Software-Entwicklung, bei denen sichergestellt wird, dass das Programm einer vorgegebenen Spezifikation entspricht. Dazu wird das Programm in eine abstrakte Darstellung überführt (zum Beispiel eine Graphdarstellung) und

dann mithilfe von formellen Logiken oder Beweisen sichergestellt, dass ein gewünschtes Verhalten eingehalten wird. Diese Methode ist jedoch sehr aufwändig, da die Spezifikationen stets an das Programm angepasst werden müssen. Besonders formelle Beweise der Funktionsweise des Programms lassen sich nur schwer automatisieren und erfordern häufiges Eingreifen des Entwicklers.

Fuzzing & Next-Gen-Analysen Beim Fuzzing werden kontinuierlich zufällige Eingaben für ein Programm erzeugt und es wird beobachtet, wie das Programm reagiert. Besonders bei sehr komplexen Programmen wie Compilern erlaubt dies ein gründliches Testen der Funktionsweise. Selbst bei Millionen von Eingaben ist es jedoch möglich, dass ein Bug vorhanden ist, wenn er von keiner dieser Eingaben ausgelöst wurde.

Ein zukünftiger Ansatz könnten zudem KI-gestützte Fuzzer sein, wie es bei dem Tool deepSURF [5] der Fall ist. Dabei wird der Fuzzer von einem LLM geführt und auf den Zielcode spezifisch angepasst. Im Gegensatz zu rein zufälligem Fuzzing könnte dies für eine höhere Erkennungsrate von Bugs sorgen.

Es ist ersichtlich, dass eine breite Auswahl an Werkzeugen zur Überprüfung von Rust-Programmen zur Verfügung steht. Jedes dieser Werkzeuge hat jedoch einen eigenen Anwendungsbereich und überdeckt sich mit dem Ziel von RUDRA nicht vollständig. Diese Tools stellen somit ein Komplement anstatt eines Ersatzes zu RUDRA dar.

5 Fazit

Es wird ersichtlich, dass RUDRA ein Erfolg war und eine große Menge an Safety-Bugs im gesamten Rust-Ökosystem aufdecken konnte. Dies beschränkte sich nicht nur auf die Standardbibliothek, sondern umfasste die gesamte Package-Registry Crates.io.

Die Evaluationen des ursprünglichen Papers ließen sich nach Behebung der anfänglichen Hürden vollumfänglich reproduzieren und verifizieren. Auffällig dabei ist, dass die eigentliche Herausforderung nicht in der Analysetechnik lag, sondern in der Konservierung der Ausführungsumgebung. Für die Nutzbarkeit wissenschaftlicher Artefakte bedeutet dies, dass die Reproduzierbarkeit weniger häufig an der Methodik selbst scheitert als an der Alterung des umgebenden Ökosystems. Sorgfältig gepinnte Umgebungen aus Images und Lockfiles sind daher ebenso entscheidend wie der Algorithmus selbst.

Darüber hinaus hatte RUDRA eine positive Auswirkung auf das Rust-Ökosystem, da es den Fokus auf bestimmte Bug-Klassen gelenkt und zu Anpassungen der Dokumentation von Unsafe-Code geführt hat. Eine Untersuchung der eingereichten Bug-Reports hat zudem ergeben, dass ein Großteil der gefundenen Fehler behoben wurde. Im direkten Vergleich von RUDRA mit anderen Analyse-Tools zeigt sich, dass die verschiedenen Werkzeuge unterschiedliche Ansätze verfolgen und sich gegenseitig ergänzen. LLMs waren zwar in der Lage, die von RUDRA identifizierten Bugs zu finden und weisen ein Potenzial für zukünftige Analysen auf, verweigerten jedoch zum Teil willkürlich die Antwort. Aufgrund der Übernahme der wichtigsten Analysen in den offiziellen Rust-Linter Clippy wurde die Weiterentwicklung des Tools mittlerweile eingestellt.

Codeverfügbarkeit

Das Repository mit den Ergebnissen unserer Arbeit befindet sich unter https://git.hhu.de/sikra111/MasterSeminar_Rudra.

Einsatz von KI

Im Rahmen dieser Ausarbeitung wurde generative künstliche Intelligenz unterstützend zur Korrektur eingesetzt.

Aufgabenverteilung

Diese Arbeit wurde als Gruppenarbeit durchgeführt. Sie lässt sich jedoch nur grob in verschiedene Verantwortungsbereiche unterteilen.

Mitglied	Aufgaben
Léon Lehnen	Verantwortlich für den Hauptbestandteil der Recherche, der Präsentationen sowie der Einordnung der Arbeit in den Kontext aktueller Forschung. Beteiligt an der Behebung von Problemen (Dependencies, Installation, Docker).
Silas Kraume	Hauptverantwortlich für die Evaluation der Reproduktionen, Auswertung und Erstellung von Plots (Datenanalyse). Gleichermaßen involviert bei den technischen Aspekten (Dependencies, Installation, Docker).

Für eine akkurate Einsicht in die individuellen Bestandteile der praktischen Arbeit lässt sich die Git-Historie des Repositories einsehen. Es ist zudem zu beachten, dass die Aufgabenverteilung nicht strikt eingehalten wurde und sich die Arbeit in vielen Bereichen überschneidet. Alle Aufgaben wurden spezifisch im Plenum besprochen und koordiniert.

Quellenverzeichnis

- [1] National Vulnerability Database [n.d.]. *CVE-2020-36317 Detail*. National Vulnerability Database. <https://nvd.nist.gov/vuln/detail/CVE-2020-36317>
- [2] National Vulnerability Database [n.d.]. *CVE-2020-36323 Detail*. National Vulnerability Database. <https://nvd.nist.gov/vuln/detail/CVE-2020-36323>
- [3] National Vulnerability Database [n.d.]. *CVE-2021-28875 Detail*. National Vulnerability Database. <https://nvd.nist.gov/vuln/detail/CVE-2021-28875>
- [4] Rust Programming Language [n.d.]. *Rust Lang Issue #81425*. Rust Programming Language. <https://github.com/rust-lang/rust/issues/81425>
- [5] Georgios Androustopoulos and Antonio Bianchi. 2025. deepSURF: Detecting Memory Safety Vulnerabilities in Rust Through Fuzzing LLM-Augmented Harnesses. arXiv:2506.15648 [cs.CR] <https://arxiv.org/abs/2506.15648>
- [6] Yechan Bae, Youngsuk Kim, Ammar Askar, Jungwon Lim, and Taesoo Kim. 2021. Rudra: Finding Memory Safety Bugs in Rust at the Ecosystem Scale. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (Virtual Event, Germany) (SOSP '21)*. Association for Computing Machinery, New York, NY, USA, 84–99. doi:10.1145/3477132.3483570
- [7] Clang Team. 2026. *AddressSanitizer — Clang Documentation*. <https://clang.llvm.org/docs/AddressSanitizer.html>
- [8] Docker, Inc.. [n.d.]. *Docker: Accelerated Container Application Development*. Docker, Inc. <https://www.docker.com/>
- [9] Steve Klabnik and Carol Nichols. [n.d.]. *References and Borrowing - The Rust Programming Language*. <https://doc.rust-lang.org/book/ch04-02-references-and-borrowing.html>
- [10] The Rayon Project Developers. [n.d.]. *Rayon: A data parallelism library for Rust*. Rust Programming Language. <https://github.com/rayon-rs/rayon>
- [11] The Rust Project Developers. [n.d.]. *Clippy: A collection of lints to catch common mistakes and improve your Rust code*. Rust Programming Language. <https://github.com/rust-lang/rust-clippy>
- [12] The Rust Project Developers. [n.d.]. *The Rust Programming Language*. <https://www.rust-lang.org/>
- [13] The Rust Project Developers. [n.d.]. *Safety comments - Standard library developers Guide*. Rust Programming Language. <https://std-dev-guide.rust-lang.org/policy/safety-comments.html>
- [14] The Rust Project Developers. 2026. *Miri: An Interpreter for Rust's Mid-level Intermediate Representation*. <https://github.com/rust-lang/miri>